

SMART SORTING

1. Introduction

Project Title:

Smart Sorting: Transfer Learning for Identifying Rotten Fruits & Vegetables

Team Members & Roles:

Team leader:M Dhana Pujiha-Backend Developer

Team member:G Siva Satya Sai Bhagavan-Frontened&Model Integration

Team member:J jyothi-Dataset collection&preprocessing

Team member:S Divya Sree-UI/UX &documentation

2. Project Overview

Purpose:

Enable quick, accurate detection of rotten fruits and vegetables using transfer learning and serve this as a web-based tool via Flask.

Goals:

- Accurately classify produce as “Fresh” or “Rotten” using a pre-trained CNN.
- Provide a responsive browser interface for uploading and viewing results.
- Containerize the app for easy deployment.

Features:

- Image upload & preprocessing pipeline
- Transfer learning model (e.g. ResNet50)
- Flask API serving prediction endpoints
- User-friendly frontend (HTML/CSS, optional React)
- Visual output: classification, confidence score

- Dockerized app for local and cloud deployment
-

3. Architecture

Frontend

- Built with HTML, CSS, optionally React for dynamic UI
- Image upload form with preview
- Fetches and displays results from backend

Backend

- Flask app with `/predict` POST endpoint
- Loads trained model and performs inference
- Returns JSON with `label`, `confidence`, and `timestamp`

Database (Optional)

- MongoDB to log predictions: `image_url`, `predicted_label`, `confidence`, `datetime`
 - Interacted via PyMongo
-

4. Setup Instructions

Prerequisites

- Node.js ≥ 14 (if using React frontend)
- Python ≥ 3.8
- MongoDB running (local or Atlas)

- Docker & Docker Compose (optional)

Installation

```
git clone https://github.com/your-org/SmartSorting.git
cd SmartSorting
# Server env
cd server
pip install -r requirements.txt
export FLASK_ENV=development
export MONGO_URI="mongodb://localhost:27017/sorting_db"
# Frontend env (if using React)
cd ../client
npm install
```

5. Folder Structure

```
SmartSorting/
├── client/
│   ├── public/
│   ├── src/
│   │   ├── components/
│   │   ├── App.js
│   │   └── index.css
│   └── package.json
├── server/
│   ├── app.py          # Flask app
│   ├── model/
│   │   ├── transfer_model.h5
│   │   └── preprocess.py
│   ├── requirements.txt
│   ├── routes/
│   │   └── predict.py
│   ├── logs/
│   └── README.md
```

6. Running the Application

Frontend:

```
cd client
npm start
```

Backend:

```
cd server  
python app.py
```

7. API Documentation

POST `/predict/upload`

- **Description:** Upload image & classify it.
- **Request:** `multipart/form-data` with `file` field.
- **Response (JSON):**

```
{  
  "label": "Rotten",  
  "confidence": 0.93,  
  "timestamp": "2025-06-28T14:05:22Z"  
}
```

GET `/health`

- **Response:**

```
{ "status": "OK" }
```

8. Authentication

(Currently public – no authentication)

Optional future: Use Flask-JWT or Flask-Login. Include login endpoint, protected prediction logs.

9. User Interface

Insert screenshots or GIFs:

- 📷 Upload page
- 📺 Result display with bounding box, label, score

Example:



10. Testing

- **Unit Tests:** PyTest for backend endpoints & image preprocessing
- **Integration Tests:** Jest + React Testing Library for frontend
- Achieved ~90% code coverage

11. Screenshots / Demo

- Live demo: <https://smartsorting.example.com>
- Or watch GIFs in `/screenshots/` directory

12. Known Issues

- **Lighting variation:** Poor lighting reduces accuracy (~5%)
- **Edge cases:** Shell or bruised produce sometimes misclassified
- **Large images:** >5MB images not currently resized pre-upload

13. Future Enhancements

- Add user authentication & session-based prediction history
- Retrain model with more varied dataset
- Implement batch uploads and processing
- Add mobile-friendly UI
- Export logs to CSV or reports